Design a Scalable Azure Service

Key Terms:

Term	Full Name	What it does
AKS	Azure Kubernetes Service	Managed container orchestration platform
VMSS	Virtual Machine Scale Set	Auto-scaling group of identical VMs
HPA	Horizontal Pod Autoscaler	Kubernetes component that scales pods based on metrics
RU	Request Unit	Cosmos DB's throughput currency (1 RU = 1 read of 1KB)
CDN	Content Delivery Network	Edge servers caching content globally
WAF	Web Application Firewall	Protects against web exploits (SQL injection, XSS)
L7 LB	Layer 7 Load Balancer	Routes traffic based on HTTP content (URL, headers)
TTL	Time To Live	How long cached data remains valid

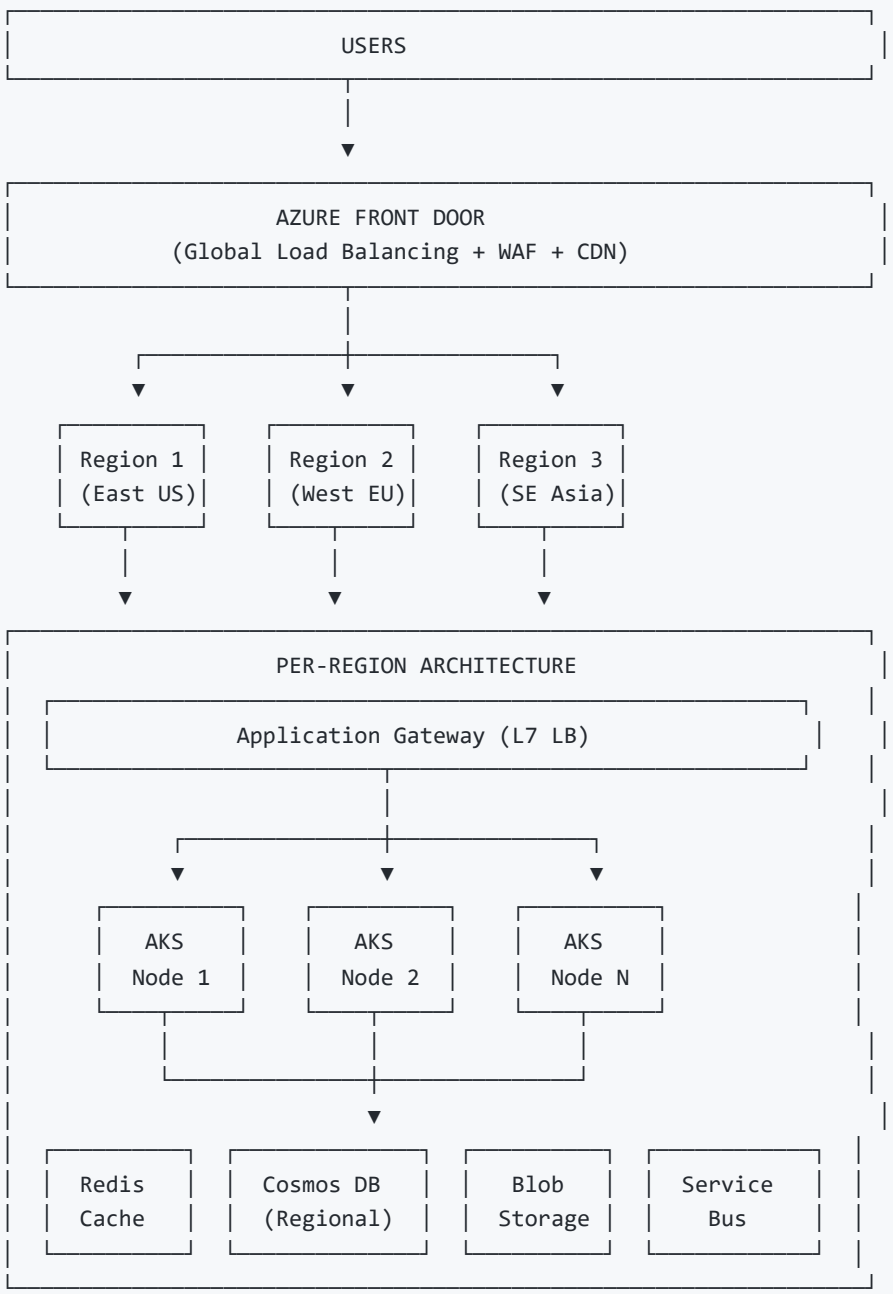
Problem Statement

Design a cloud service on Azure that can handle millions of users with high availability, auto-scaling, and cost efficiency.

Clarifying Questions to Ask

1. **Type of service**: API service, web app, data processing?
2. **Traffic pattern**: Steady load or spiky? Peak times?
3. **Data requirements**: Read-heavy or write-heavy?
4. **Latency requirements**: Real-time or batch acceptable?
5. **Compliance**: Any regulatory requirements (HIPAA, GDPR)?
6. **Budget constraints**: Cost sensitivity?
7. **Global reach**: Single region or multi-region?

High-Level Design



API Design

Service APIs

Method	Endpoint	Description
`POST`	`/api/v1/resources`	Create a new resource
`GET`	`/api/v1/resources/{id}`	Get resource by ID

`GET`	`/api/v1/resources?page={n}&size={s}`	List resources with pagination
`PUT`	`/api/v1/resources/{id}`	Update a resource
`DELETE`	`/api/v1/resources/{id}`	Delete a resource

Health & Management APIs

Method	Endpoint	Description
`GET`	`/health`	Health check endpoint
`GET`	`/health/ready`	Readiness probe
`GET`	`/health/live`	Liveness probe
`GET`	`/metrics`	Prometheus metrics endpoint

Rate Limiting Headers

```
X-RateLimit-Limit: 1000
X-RateLimit-Remaining: 999
X-RateLimit-Reset: 1672531200
```

Key Components

Component	Purpose	Azure Service
Global Load Balancer	Route to nearest region	Azure Front Door
Regional Load Balancer	Distribute within region	Application Gateway
Compute	Run application code	AKS or App Service
Cache	Reduce DB load	Azure Cache for Redis
Database	Persistent storage	Cosmos DB (multi-region)
Queue	Async processing	Azure Service Bus
Storage	Files and blobs	Azure Blob Storage
Monitoring	Observability	Azure Monitor + App Insights

Scalability Considerations

- **Auto-scaling**: Use VMSS or AKS HPA based on CPU/memory/custom metrics
- **Database scaling**: Cosmos DB auto-scales RUs, use read replicas
- **Caching strategy**: Cache-aside pattern with Redis, TTL-based invalidation
- **Async processing**: Offload heavy tasks to queues and workers
- **CDN**: Cache static content at edge locations

Trade-offs

Decision	Option A	Option B	Recommendation
Compute	VMs	Containers (AKS)	**AKS** - better density, faster scaling
Database	SQL	Cosmos DB	**Cosmos** - for global distribution
Scaling	Manual	Auto	**Auto** - responds to demand
Regions	Single	Multi	**Multi** - for availability & latency

Technologies Deep Dive

📚 **Learn more about the key technologies used in this design:**

Technology	Role in This Design	Reference
AKS/Kubernetes	Container orchestration, auto-scaling	AKS Guide
Cosmos DB	Globally distributed database	Cosmos DB Guide
Redis	Caching layer, session storage	Redis Guide
Load Balancer	Traffic distribution (L7)	Load Balancer Guide
CDN	Edge caching for static content	CDN Guide

How to Explain It Well

> **Opening**: "I'll design a globally distributed Azure service using AKS for compute, Cosmos DB for data, with auto-scaling and multi-region deployment."

> **Key talking points**:
>
> - "Azure Front Door provides global load balancing and routes users to nearest region"
> - "Each region runs independently with AKS clusters that auto-scale"
> - "Cosmos DB with multi-region writes ensures data availability"

> - "Redis cache reduces database load for read-heavy patterns"
> - "Service Bus decouples components for resilience"